

Dossier de conception — Mardik v2

Livrable d'entrée du brief « Du prototype au produit ». Les décisions sont prises ici ; le code les applique, il ne les redéfinit nulle part.

1. Synthèse du besoin

Fonctionnel — ce que la v2 doit faire. Analyser un contrat commercial **quelle que soit sa longueur** (30 à 40 pages, là où la v1 oublie la fin : résiliation, pénalités, droit applicable) et accompagner chaque clause relevée d'un **indice de fiabilité**, par clause et pour l'ensemble, pour que le juriste sache quand relire et quand faire confiance.

Contraintes — ce qui borne la solution. Délai de réponse inférieur à 8 s pour 95 % des analyses, contrats longs compris. Coût moyen inférieur à 0,15 € par analyse, pour ~400 contrats/mois. Disponibilité continue de la v1 : le client interne `client_v1` est intégré à l'outil de gestion documentaire et **ne sera pas refait** — son contrat d'API est intangible. Retour arrière immédiat en cas de problème, sans attendre un correctif.

Hors périmètre — assumé et argumenté. La **traduction** des contrats en anglais et le **chatbot** évoqués en fin d'expression de besoin sont écartés de cette livraison. Ce ne sont pas des variantes de la version du modèle d'analyse mais deux produits distincts : ils n'ont ni le même contrat d'API, ni les mêmes critères de qualité, ni le même jeu d'évaluation. Les inclure rendrait la v2 inévaluable — on ne peut pas noter un chatbot avec des contrats annotés en clauses — et donc non livrable par une chaîne à gate bloquant, ce qui est l'exigence principale du CTO. Ils feront l'objet d'une expression de besoin propre.

Est également hors périmètre la **réécriture de la v1** : elle reste servie telle quelle, avec sa troncature, jusqu'à ce que la v2 ait pris 100 % du trafic.

2. Langage du domaine

Le vocabulaire du métier devient le vocabulaire du code et de l'interface. Les deux termes que les juristes emploient pour décrire ce qui ne va pas :

Terme métier	Sens pour le juriste	Dans le système
contrat tronqué	« l'outil n'a pas lu la fin »	<code>tronque: true</code> dans la réponse v1, <code>tronques</code> au tableau de bord
analyse non fiable	« je ne sais pas si je peux m'en servir »	<code>confiance_globale</code> < 0,70
fiabilité	ce que l'interface montre au juriste	libellé du frontend ; <code>confiance</code> dans l'API
clause	un type de disposition juridique	<code>TYPES_CLAUSES</code> , vocabulaire fermé de 14 libellés

Ce que « score de confiance » veut dire pour un juriste qui décide. Pas une probabilité, pas une note de qualité : une **recommandation d'action**. L'API renvoie un nombre, l'interface le traduit en trois décisions possibles — *exploitable* ($\geq 0,85$), à *vérifier* ($0,70-0,85$), à *repandre* ($< 0,70$). C'est pourquoi le tableau de bord découpe la distribution sur ces mêmes bornes et non en dixièmes réguliers : un histogramme « neutre » n'aurait rien dit de ce qu'il faut faire.

3. Choix d'outils et de conception non triviaux

Les outils familiers (FastAPI, pytest, ruff, uv, Docker, GitHub Actions, OpenTelemetry, structlog) ne sont pas justifiés ici. Les décisions qui méritent une justification :

Un registre d'artefacts en système de fichiers, pas MLflow ni un registre distant. Une version de Mardik est un bundle de configuration de quelques kilo-octets, pas un fichier de poids. Un dossier par version avec son `manifest.json` donne l'immuabilité, l'horodatage, l'empreinte et la traçabilité attendues, se lit sans service tiers, se versionne et se sauvegarde avec le reste. MLflow apporterait une interface et un coût d'exploitation pour un problème qu'on n'a pas.

Un routage canary par compteur cyclique, pas par tirage aléatoire. À 30 %, l'aléatoire donne « environ » 30 % : sur 40 requêtes de démonstration il peut n'en router aucune et faire croire à une panne du canary. Le compteur donne exactement 30 sur 100, immédiatement, ce qui rend la montée en charge observable et reproductible. Contrepartie assumée : pas de session collante — acceptable tant que les analyses sont indépendantes, à revoir si le chatbot arrive.

Un score de confiance composite, pas la confiance déclarée par le modèle. Un LLM est tout aussi sûr de lui sur une citation inventée. Le score multiplie la confiance déclarée par l'**ancrage** de l'extrait — vérification textuelle faite *sans* le modèle, donc sans partager son biais. Une citation fabriquée par un modèle sûr à 0,95 tombe à 0,33 ; une moyenne l'aurait laissée à 0,48, un score ambigu. Voir `app/pipeline/confiance.py`.

Un mode MOCK avec fixtures enregistrées pour la CI. Douze contrats $\times$ une vingtaine de sections à chaque fusion de chaque équipe coûteraient plus que le service. La CI rejoue des réponses enregistrées : déterministe, gratuite, rapide, donc légitime pour bloquer une PR. Le modèle réel n'est appelé que par le gate de release, sur tag.

Des appels de sections parallélisés. Un contrat de 40 pages produit ~ 35 appels ; en série il tiendrait ~ 35 s, bien au-delà des 8 s contractuelles. La concurrence est un paramètre **du bundle** et non du code : elle fait partie de la version, donc elle se rollback avec elle.

4. Contrat d'API

4.1 /v1/analyse — intangible

```
POST /v1/analyse {"texte": "<contrat>"}
→ 200 {"clauses": ["résiliation", ...], "modele": "...",
      "version": "v1.0.0", "tronque": true|false}
→ 422 corps invalide (detail explicite)
→ 503 fournisseur LLM indisponible (detail explicite)
```

Rien n'a le droit de changer : ni le chemin, ni le corps de requête, ni les champs de réponse, ni leur type. Le client historique n'est pas refait. La non-régression est vérifiée par `scripts/client_v1.py`, exécuté comme un test à part entière à chaque fusion.

4.2 /v2/analyse — la nouvelle version

```
POST /v2/analyse {"texte": "<contrat>", "contrat_id": "c07" (optionnel)}
→ 200 {
  "clauses": [
    {"type": "résiliation",           // l'un des 14 libellés fermés
      "extrait": "...",              // citation littérale du contrat
      "confiance": 0.91,             // score composite, 0-1
      "sections": [3, 17]}          // où la clause a été vue
    ],
  "confiance_globale": 0.87,         // 0,7 × moyenne + 0,3 × minimum
  "modele": "...", "version": "v2.0.0",
  "sections": 35,                   // nombre de sections analysées
  "appels_llm": 35, "latence_ms": 5230.4, "cout_eur": 0.031
}
```

4.3 /analyse — la gateway

Même corps de requête, réponse de la version servie, en-tête `X-Mardik-Version` indiquant **quelle version a répondu**. C'est l'entrée des nouveaux clients et du frontend ; `/v1` et `/v2` restent accessibles en direct.

4.4 Codes d'erreur

Code	Cas	Garantie
422	corps invalide, texte < 20 caractères	<code>detail</code> nommant le champ
503	fournisseur LLM injoignable, aucune version active, stratégie inconnue	<code>detail</code> nommant la cause
500	erreur interne	<code>detail</code> explicite et journalisé — jamais de 500 muet

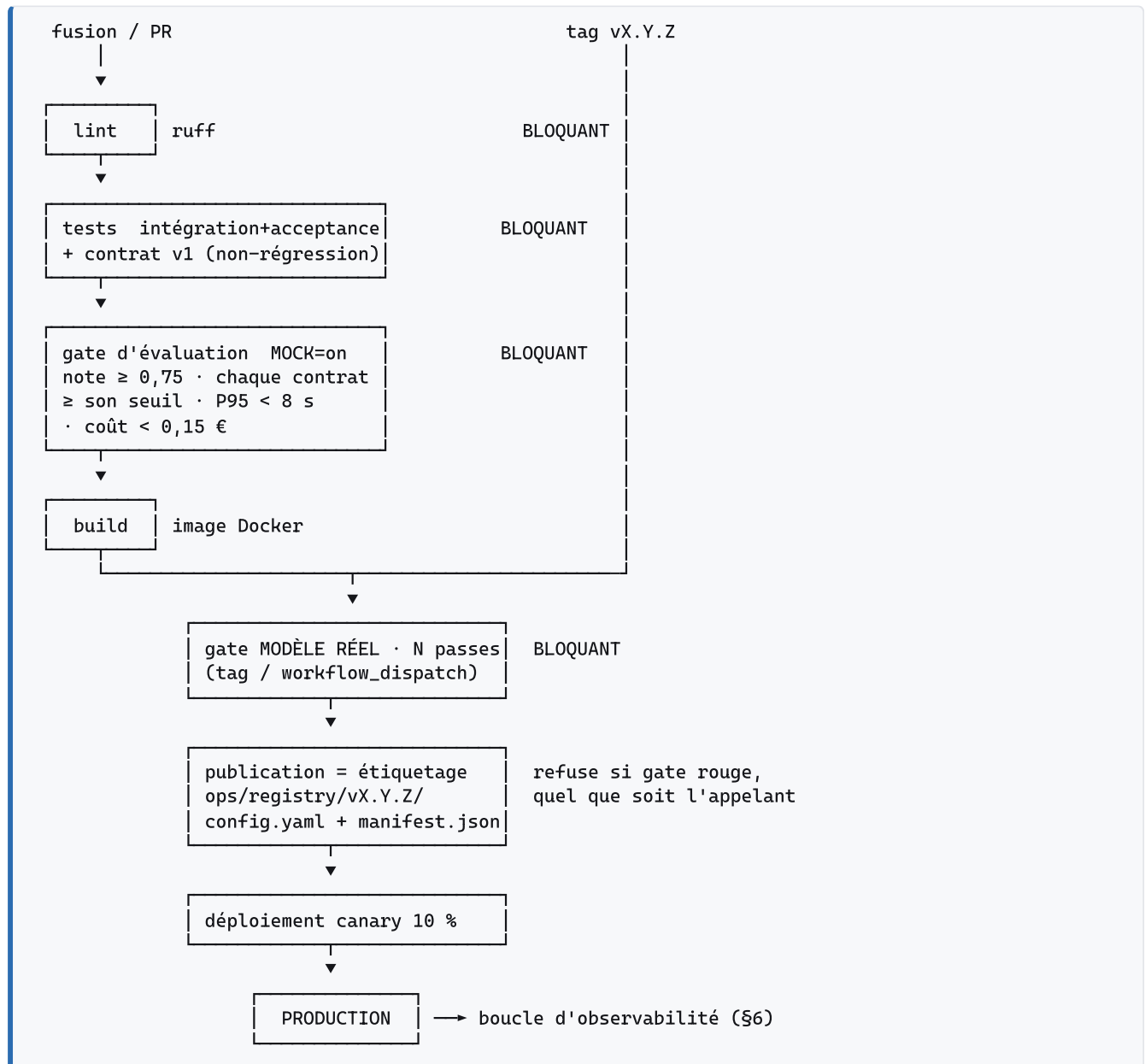
4.5 Versionnage

Une version est le **bundle** `models/vX/config.yaml` : modèle de base, prompt, paramètres d'échantillonnage, schéma de sortie, stratégie. Le code lit le bundle et n'embarque aucune de ces valeurs.

- **MAJEUR** — le contrat d'API change ; un client doit être adapté (v1 → v2).

- **MINEUR** — la sortie change sans casser le contrat (prompt, modèle, stratégie, paramètres). Gate obligatoire.
- **CORRECTIF** — ni le contrat ni la sortie attendue ne changent.

5. Schéma de la chaîne LLMOps

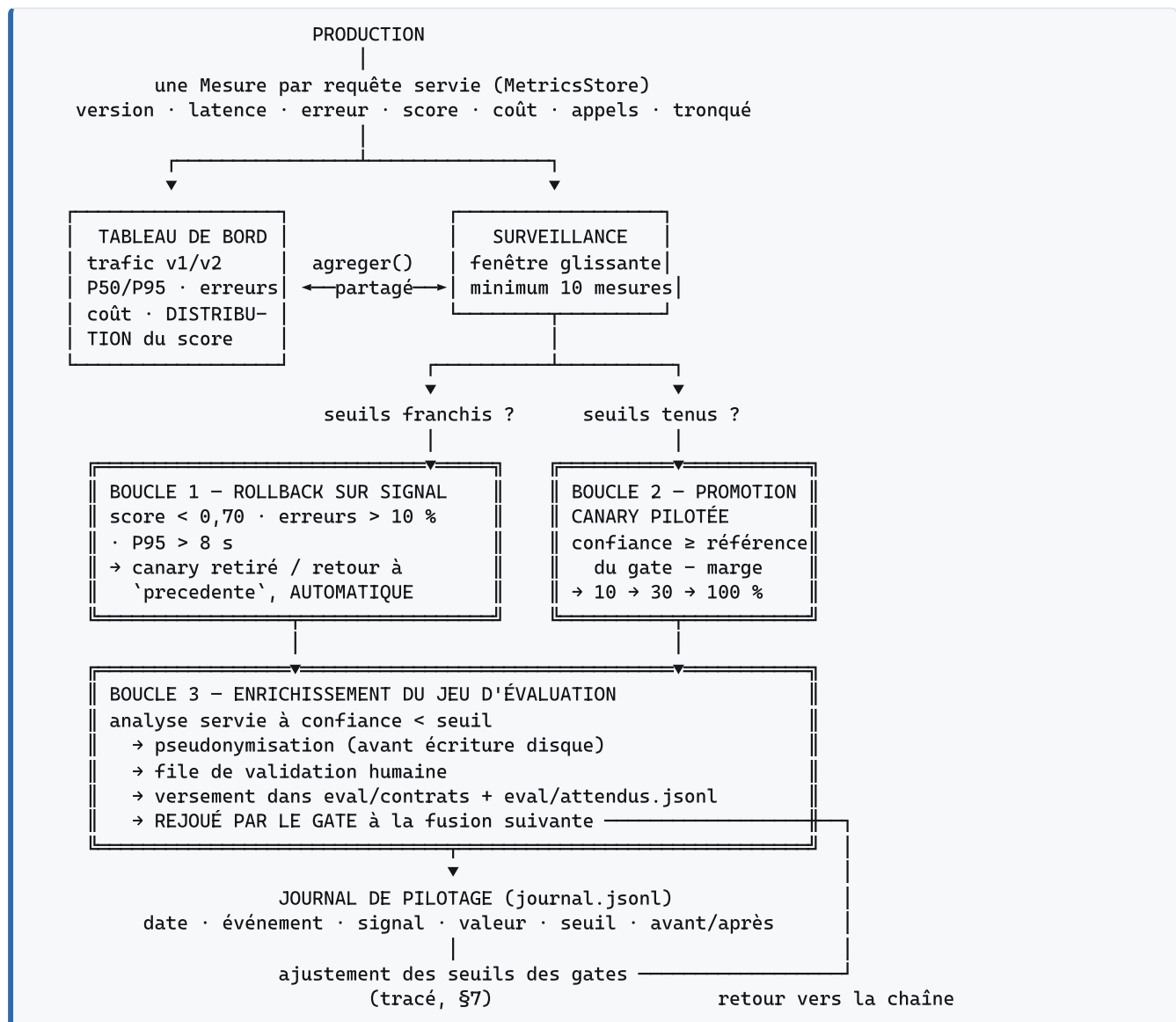


Ce qui bloque et pourquoi. Les gates MOCK bloquent chaque PR parce qu'ils sont déterministes et gratuits : un échec y désigne une régression de code, pas l'humeur du modèle. Le gate sur modèle réel bloque la publication mais ne tourne pas sur chaque PR, pour des raisons de coût. `ops.deploy.pu` rejoue le gate et refuse l'étiquetage s'il échoue : **il n'existe aucun chemin vers le registre qui contourne la vérification**, ni par la CI, ni par la console.

Seuils initiaux et leur provenance. `note ≥ 0,75` : en dessous, un quart des clauses attendues manquent, le juriste relit tout. `P95 < 8 s` et `coût < 0,15 €` : repris tels quels de `besoin_client.md`, ce sont des engagements, pas des réglages. Les seuils par contrat (0,75, et 0,80 pour les contrats longs) sont plus

exigeants là où la v1 échouait : c'est le progrès qu'on livre, il doit être mesuré là. Ces valeurs sont **provisoires par construction** — le chantier 2 les ajuste sur les distributions observées, et chaque ajustement est tracé (§7).

6. Schéma de la boucle d'observabilité



Pourquoi la distribution et pas seulement la moyenne. Dix analyses à 0,95 et dix à 0,45 donnent la même moyenne que vingt analyses à 0,70. Dans le premier cas la moitié du travail est à refaire, dans le second aucun n'est franchement mauvais. La moyenne ne distingue pas ces deux situations ; la distribution si, et c'est elle qui dit s'il faut agir.

Vraie dérive ou bruit. Trois garde-fous cumulés : une **fenêtre glissante** (120 s par défaut), un **minimum de mesures** (10, et 30 pour une promotion) en dessous duquel la boucle s'abstient, et une **marge** sur le seuil de promotion. Sans le minimum, trois requêtes malheureuses au redémarrage suffiraient à déclencher un rollback — et une boucle qui se déclenche à tort finit désactivée, ce qui coûte aussi les vraies détections.

Dérive par rapport à quoi. La référence est la confiance mesurée **par le gate, au moment où la version est sortie de la chaîne**, figée dans son manifeste (`confiance_moyenne`). Pas une moyenne glissante de production : une référence qui glisse absorbe lentement la dérive qu'elle est censée détecter et finit par la valider.

Qui décide quoi. La *descente* est automatique — attendre un humain pendant que la production dérive coûte des analyses fausses. La *montée* est automatique aussi, mais seulement d'un palier à la fois et sous condition de référence ; le passage à 100 % reste une décision humaine (`promouvoir`), parce qu'aucun signal disponible ne mesure la justesse juridique. Une boucle qui promeut seule sur des indicateurs techniques promeut un modèle rapide et pas cher, pas un modèle juste.

7. Tableau de pilotage

Signal	Seuil	Rétroaction déclenchée	Trace
confiance moyenne (canary)	< 0,70 sur ≥ 10 mesures / 120 s	rollback automatique : canary retiré ou retour à <code>precedente</code>	<code>rollback</code> — motif, <code>avant / apres</code>
taux d'erreur	> 10 %	rollback automatique	<code>rollback</code> — motif
latence P95	> 8 000 ms	rollback automatique	<code>rollback</code> — motif
confiance moyenne ≥ référence du gate – 0,05, sur ≥ 30 mesures	tenu	promotion au palier suivant (10 → 30 → 100 %)	<code>promotion_canary</code> — signal, valeur, seuil, <code>avant / apres</code>
confiance moyenne < référence du gate – 0,05	franchi	promotion refusée , canary maintenu	<code>promotion_refusee</code> — motif, valeur, seuil
confiance d'une analyse servie	< 0,70 (ajustable)	capture du cas, pseudonymisé, en file de validation	<code>capture_faible_confiance</code> — valeur, seuil, substitutions
validation humaine d'un cas	—	versement au jeu d'éval, rejoué au gate suivant	<code>enrichissement_jeu_evaluation</code> — auteur, clauses retenues
rejet humain d'un cas	—	cas écarté définitivement	<code>capture_rejetee</code> — auteur, motif
distribution observée (P10/P50/P90)	décision d'exploitation	ajustement d'un seuil de pilotage	<code>ajustement_seuil</code> — auteur, motif, <code>avant / apres</code>
gate d'évaluation	note < seuil, ou P95/coût hors contrainte	livraison bloquée	<code>ErreurDeployment</code> , build rouge

Toutes les traces sont des lignes JSON dans `ops/registry/journal.jsonl` , horodatées, avec le signal déclencheur et l'état avant/après. Le tableau de bord en affiche les cinq dernières ; le fichier est la source.

8. Questions ouvertes et réponses retenues

Le canary sépare-t-il deux images du service, ou deux versions du modèle ? Deux versions du modèle.

L'image est la même : c'est le bundle servi qui change, choisi par la gateway à chaque requête selon `index.json`. C'est ce qui permet un rollback sans redéploiement et un canary sans orchestrateur.

Deux boucles ou trois ? Les trois sont livrées et testées : rollback sur signal, promotion canary pilotée, enrichissement du jeu d'évaluation. Le brief en exige « au moins deux » en démonstration ; les livrables en listent trois. Nous livrons les trois pour ne pas avoir à choisir.

M1 / M3 sont référencés dans les étapes du chantier 1 mais définis nulle part dans le texte disponible. Nous les avons lus comme « le contrat d'API v1 ne change pas » et « les erreurs restent explicites », deux exigences par ailleurs tenues et testées. Signalé une fois, sans y revenir.

La v1 est-elle figée au point qu'on ne puisse pas l'observer ? Non : son *contrat d'API* est intangible, son instrumentation ne l'est pas. La v1 émet déjà des spans et des `Mesure` (héritage de la phase de remédiation), ce qui est indispensable — sans mesures v1, la part de trafic v1/v2 et la comparaison des latences seraient impossibles.

Où vivent les secrets ? Dans l'environnement (`.env` en local, variables du service en hébergement), jamais dans le bundle ni dans le registre. Le bundle référence `${LLM_MODEL}` et le client lit `AZURE_AI_API_KEY` à l'appel. Un manifeste de version est destiné à être lu et archivé : il ne doit contenir que de la configuration publiable.